Príloha A

Použitie LLM a AI agentov

V rámci vypracovania projektu boli využité nástroje založené na veľkých jazykových modeloch (LLM). Tieto nástroje slúžili ako podpora pri vybraných činnostiach súvisiacich s návrhom, implementáciou a dokumentáciou projektu.

V nasledujúcich častiach sú jednotlivé nástroje uvedené samostatne spolu s účelom ich využitia, použitými promptmi, relevantnými výstupmi a spôsobom ich využitia v projekte.

1. ChatGPT

1.1. Opis nástroja

Nástroj:

ChatGPT

Model:

GPT-6

Účel použitia:

Vysvetlenie pojmov, práca s .tex dokumentmi, štruktúrovanie informácií.

1.2. Použité prompty a relevantné výstupy

Pri práci s nástrojom ChatGPT boli použité nasledujúce prompty a získané výstupy:

Vysvetli ako funguje LinearSVC a ako ho správne implementovať?

1. Čo je LinearSVC?

LinearSVC (Linear Support Vector Classification) je algoritmus strojového učenia, ktorý sa používa na klasifikáciu údajov.

Jeho úlohou je rozdeliť údaje do jednotlivých tried pomocou lineárnej rozhodovacej hranice.

Napríklad:

- Rozpoznávanie e-mailov: spam alebo nie spam.
- Klasifikácia textov: pozitívny alebo negatívny.
- Rozpoznávanie typov rastlín podľa ich vlastností.

2. Ako LinearSVC funguje?

Algoritmus sa snaží nájsť takú priamku (pri dvoch vlastnostiach) alebo nadrovinu (pri viacerých vlastnostiach), ktorá čo najlepšie oddeľuje jednotlivé triedy.

Hlavný princíp je maximalizácia marginu – vzdialenosti medzi rozhodovacou hranicou a najbližšími údajmi jednotlivých tried, pričom algoritmus pripúšťa aj určité chyby klasifikácie.

Rozhodovanie sa vykonáva pomocou funkcie:

$$f(x) = w^T x + b$$

Kde:

- x – vstupné údaje.
- w – váhy jednotlivých vlastností.
- b – posunutie rozhodovacej hranice.

Pri binárnej klasifikácii kladná hodnota znamená jednu triedu, záporná druhú.

Dôležité: LinearSVC používa lineárnu rozhodovaciu hranicu, preto nemusí byť vhodný pre zložité nelineárne vzťahy.

3. Najdôležitejšie parametre

- `C` – Určuje silu regularizácie. Menšie C znamená silnejšiu regularizáciu a väčšiu toleranciu chýb.
- `max_iter` – Maximálny počet iterácií pri trénovaní.
- `dual` – Určuje spôsob optimalizácie. Hodnota `"auto"` vyberie vhodný spôsob automaticky.
- `class_weight` – Umožňuje zohľadniť nevyvážené triedy.
- `random_state` – Pomáha zabezpečiť reprodukovateľnosť výsledkov tam, kde algoritmus používa náhodnosť.

Predvolená hodnota `C=1.0` je vhodný východiskový bod.

4. Správna implementácia v Pythone

Na implementáciu sa používa knižnica `scikit-learn`.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import LinearSVC

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = make_pipeline(
    StandardScaler(),
    LinearSVC(C=1.0, max_iter=10000)
)

model.fit(X_train, y_train)

print("Presnosť:", model.score(X_test, y_test))
```

Vysvetlenie implementácie

1. **Načítanie údajov:** `load_iris()` poskytuje údaje o rastlinách a ich triedach.
2. **Rozdelenie údajov:** `train_test_split()` rozdelí údaje na 80 % tréningových a 20 % testovacích.
3. **Štandardizácia:** `StandardScaler()` upraví číselné vlastnosti na porovnateľnú mierku.
4. **Vytvorenie modelu:** `LinearSVC()` vytvorí lineárny klasifikátor.
5. **Trénovanie:** `fit()` naučí model rozlišovať triedy.
6. **Vyhodnotenie:** `score()` vypočíta presnosť klasifikácie na testovacích údajoch.

Použitie `Pipeline` zabezpečuje, že štandardizácia sa naučí iba z tréningových údajov, čím

sa predchádza úniku informácií z testovacích údajov.

5. Na čo si dávať pozor?

Pri správnej implementácii je dôležité:

- **Štandardizovať údaje**, pretože LinearSVC je citlivý na rozdielne mierky vlastností.
- **Vyhodnocovať model na testovacích údajoch**, nie iba na tréningových.
- **Správne nastaviť parameter C**, pretože ovplyvňuje kompromis medzi jednoduchším modelom a chybami klasifikácie.
- **Použiť** `class_weight="balanced"`, ak sú triedy výrazne nevyvážené, a sledovať aj precision, recall alebo F1-score.

6. Rozdiel medzi SVC a LinearSVC

- **Rozhodovacia hranica:** LinearSVC používa lineárnu rozhodovaciu hranicu, zatiaľ čo SVC podporuje lineárne aj nelineárne kernely.
- **Rýchlosť:** LinearSVC je zvyčajne efektívnejší pre veľké datasety, zatiaľ čo SVC môže byť pri veľkých datasetoch pomalší.
- **Viac tried:** LinearSVC rieši štandardne viac tried metódou One-vs-Rest, zatiaľ čo SVC používa One-vs-One.
- **Pravdepodobnosti:** LinearSVC neposkytuje `predict_proba()`, zatiaľ čo SVC môže poskytovať pravdepodobnosti pri nastavení `probability=True`.

Zhrnutie

LinearSVC je algoritmus klasifikácie, ktorý hľadá optimálnu lineárnu hranicu medzi triedami s čo najväčším marginom a primeranou penalizáciou chýb.

Pri implementácii je najdôležitejšie správne rozdeliť údaje, vykonať štandardizáciu, natrénovať model a vyhodnotiť jeho presnosť na testovacích údajoch.

Ako opraviť chybu „$ expected“ v LaTeXu?

Ako opraviť chybu „$ expected“ v LaTeXu?

Chyba **„Missing $ inserted“** alebo **„$ expected“** vzniká vtedy, keď LaTeX očakáva matematický režim, ale text alebo príkaz je zapísaný nesprávne.

1. Najčastejšie príčiny chyby

- **Znak podčiarknutia _** – LaTeX ho používa na zápis dolných indexov v matematickom režime.
- **Znak ^** – používa sa na zápis horných indexov.
- **Neuzavretý matematický režim** – chýba ukončovací znak `$`.
- **Špeciálne znaky** – niektoré znaky majú v LaTeXu osobitný význam a musia byť správne zapísané.

2. Ako chybu opraviť?

Príklad 1: Podčiarknutie v texte

Nesprávne:

```
random_state
```

Správne:

```
random\_state
```

Príklad 2: Podčiarknutie v názve parametra

Nesprávne:

```
\texttt{class_weight}
```

Správne:

```
\texttt{class\_weight}
```

Príklad 3: Neuzavretý matematický režim

Nesprávne:

```
Hodnota je $x + 5.
```

Správne:

```
Hodnota je $x + 5$.
```

3. Ako nájsť chybu v dokumente?

1. Pozri sa na číslo riadku, ktoré LaTeX uvádza pri chybe.
2. Skontroluj daný riadok a niekoľko predchádzajúcich riadkov.
3. Vyhľadaj znaky podčiarknutia, horného indexu alebo neuzavreté znaky matematického režimu.
4. Oprav nesprávny zápis.
5. Znova skompiluj dokument.

4. Ako predísť tejto chybe?

Pri písaní technických dokumentov používaj správny zápis špeciálnych znakov.

Napríklad:

- Podčiarknutie: `_` → `\_`
- Percento: `%` → `\%`
- Ampersand: `&` → `\&`
- Mriežka: `#` → `\#`

Pri zápise názvov premenných z Pythonu, napríklad `max_iter`, `random_state` alebo `class_weight`, používaj `\texttt{}` a správne ošetri podčiarknutia.

Zhrnutie

Chyba **„$ expected“** najčastejšie vzniká nesprávnym použitím špeciálnych znakov alebo matematického režimu.

Pri jej oprave je potrebné skontrolovať problémový riadok, správne zapísať špeciálne znaky a overiť uzavretie matematického režimu.

Skontroluj celý text dokumentu z hľadiska pravopisu, gramatiky, lexiky a štylistiky. Oprav všetky zistené jazykové chyby bez zmeny odborného významu textu a zachovaj pôvodnú štruktúru dokumentu.

Bola vykonaná jazyková kontrola celého dokumentu. Boli identifikované a opravené pravopisné, gramatické, lexikálne a štylistické nedostatky. Odborný obsah a pôvodná štruktúra dokumentu zostali zachované.

1.3. Spôsob využitia výstupov

Relevantný výstup 1: Z prvého výstupu boli získané základné teoretické poznatky o

fungovaní algoritmu LinearSVC, jeho hlavných parametroch a správnej implementácii. Tieto informácie boli využité na lepšie pochopenie algoritmu a jeho použitia v projekte. Správnosť informácií bola overená porovnaním s dokumentáciou knižnice scikit-learn.

Relevantný výstup 2: Druhý výstup bol využitý na identifikáciu a opravu chyby „Missing $ inserted“ v LaTeXu. Na základe získaných informácií boli opravené nesprávne zapísané špeciálne znaky a upravená syntaxa dokumentu. Správnosť opráv bola overená opätovnou kompiláciou dokumentu.

Relevantný výstup 3: Tretí výstup bol využitý na jazykovú korektúru celého dokumentu. Na základe získaných odporúčaní boli opravené pravopisné, gramatické a lexikálne chyby a upravené niektoré štylistické formulácie. Správnosť úprav bola overená následnou kontrolou textu.

2. Claude

2.1. Opis nástroja

Nástroj:

Claude

Model:

[Sonnet 5.5]

Účel použitia:

[Podpora pri riešení praktickej časti projektu]

2.2. Použité prompty a relevantné výstupy

Pri práci s nástrojom Claude boli použité nasledujúce prompty a získané výstupy:

Prompt 1:

Ako správne implementovať TF-IDF v kombinácii s LinearSVC na klasifikáciu textových údajov v Pythone?

Relevantný výstup 1:

Ako správne implementovať TF-IDF + LinearSVC

Kľúčové zásady:

1. **Použite Pipeline**, aby sa slovník a IDF váhy učili len z trénovacích dát (žiadny únik dát).
2. **Znakové n-gramy** sa zapínajú parametrom `analyzer='char'` (alebo `'char_wb'`) a `ngram_range`.
3. **LinearSVC nastavte explicitne** (`dual`, `max_iter`, `random_state`) kvôli reprodukovateľnosti.
4. **Vyhodnoťte pomocou Macro-F1**, pretože triedy sú nevyvážené.

Základná implementácia

```
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import f1_score, classification_report

SEED = 2024

# 1. Nacitanie dat
train = pd.read_csv(
    "data/incidents_train.csv", index_col=0
)
valid = pd.read_csv(
    "data/incidents_valid.csv", index_col=0
)

# 2. Definicia pipeline
def make_pipeline():
    return Pipeline([
        ("tfidf", TfidfVectorizer(
            analyzer="char",
            ngram_range=(2, 5),
            strip_accents="unicode",
            lowercase=True,
            max_df=0.5,
            min_df=5,
            sublinear_tf=True,
        )),
        ("clf", LinearSVC(
            C=1.0,
            dual=True,
            max_iter=10000,
            random_state=SEED,
        )),
    ])

# 3. Samostatny model pre kazdy stitok
labels = [
    "hazard-category",
    "product-category",
    "hazard",
    "product"
]

models, scores = {}, {}

for label in labels:
    pipe = make_pipeline()
    pipe.fit(train["title"], train[label])
    pred = pipe.predict(valid["title"])
    models[label] = pipe
    scores[label] = f1_score(
        valid[label],
        pred,
        average="macro",
        zero_division=0
    )
    print(
        f"{label}: macro-F1 = {scores[label]:.3f}"
    )
```

Nastavenie znakových n-gramov

- **`analyzer='char'`** – n-gramy cez celý text vrátane medzier medzi slovami. Dobrá voľba pre krátke texty a preklepy.
- **`analyzer='char_wb'`** – n-gramy len vnútri hraníc slov (s paddingom medzerami). Čistejšie príznaky, často podobné výsledky.
- **`ngram_range=(2,5)`** – minimálna a maximálna dĺžka n-gramu. Typicky skúšať (2,5), (3,5), (3,6).
- **`min_df`** – minimálny počet dokumentov. Odporúčaná hodnota 2–5. Znižuje šum

a veľkosť slovníka.

- **`max_df`** – maximálny podiel dokumentov. Odporúčaná hodnota 0,5–0,9.
- **`sublinear_tf=True`** – logaritmické škálovanie frekvencie. Väčšinou pomáha.
- **`max_features`** – horný limit veľkosti slovníka. Užitočné pri pamäťových obmedzeniach.

Väčší rozsah `ngram_range` zvyšuje počet príznakov (pamäť a čas), zachytáva však viac morfologických a pravopisných variantov.

Ladenie hyperparametrov (to, čo v práci chýba)

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "tfidf__ngram_range": [
        (2, 5), (3, 5), (3, 6)
    ],
    "tfidf__min_df": [2, 5],
    "clf__C": [0.1, 0.5, 1.0, 2.0],
    "clf__class_weight": [None, "balanced"],
}

search = GridSearchCV(
    make_pipeline(),
    param_grid,
    scoring="f1_macro",
    cv=5,
    n_jobs=-1,
)

search.fit(
    train["title"],
    train["hazard-category"]
)

print(search.best_params_, search.best_score_)
```

Pri štítkoch `hazard` a `product` s tisíckami tried môže byť 5-násobná krížová validácia problematická (triedy s 1–2 vzorkami).

Vtedy použite jedno pevné rozdelenie, alebo `PredefinedSplit` s oficiálnou validačnou množinou.

Praktické odporúčania

- **Kombinácia title + text:**

  ```
  train["title"] + " " + train["text"]
  ```

  alebo dva vektorizátory cez `ColumnTransformer` (vhodné aj slovné n-gramy spolu so znakovými cez `FeatureUnion`).

- **Reprodukovateľnosť:**

  Fixný `random_state`, uloženie verzií knižníc:

  ```
  pip freeze > requirements.txt
  ```

- **Konvergencia:**

  Ak sa objaví `ConvergenceWarning`, zvýšte `max_iter`.

- **Nevyváženosť tried:**

  Vyskúšajte `class_weight='balanced'`, ale vyhodnoťte to na oboch množinách (ako ukazuje práca, prínos nemusí byť konzistentný).

- **Pravdepodobnosti:**

`LinearSVC` nemá `predict_proba`. Ak ich potrebujete, použite:

`CalibratedClassifierCV(LinearSVC())`

- **Konzistencia štítkov:**

  Po predikcii `hazard` môžete `hazard-category` odvodiť z mapovania hazard na kategóriu získaného z trénovacích dát.

Zhrnutie

Správna implementácia zahŕňa použitie `Pipeline` s TF-IDF, `analyzer='char'` a `ngram_range=(2,5)`, ako aj modelu `LinearSVC` s explicitne nastavenými parametrami.

Model sa trénuje na trénovacej množine, parametre sa ladia na validačnej množine a záverečné vyhodnotenie sa vykonáva pomocou Macro-F1 na testovacej množine.

Najväčší priestor na zlepšenie práce je v ladení parametra C a n-gramov, využití textu okrem titulku a v dohľadaní anomálie bootstrap intervalu pri logistickej regresii.

Prompt 2:

[Ako správne vyhodnotiť klasifikačný model pomocou metrík Macro-F1 a Micro-F1 v knižnici scikit-learn?]

Relevantný výstup 2:

Vyhodnotenie klasifikačného modelu pomocou Macro-F1 a Micro-F1

1. Rozdiel medzi metrikami

- **Macro-F1** vypočíta F1 samostatne pre každú triedu a spriemeruje ich bez váhovania. Každá trieda má rovnakú váhu, takže metrika penalizuje slabé výsledky na zriedkavých triedach. Je to hlavná metrika súťaže SemEval-2025 Task 9.
- **Micro-F1** spočíta globálne TP, FP a FN cez všetky vzorky a až potom počíta F1. Dominujú jej frekventované triedy. Pri klasifikácii s jedným štítkom na vzorku je totožná s presnosťou (accuracy).
- **Weighted-F1** (doplnok) je priemer F1 vážený počtom vzoriek v triede.

Vo vašej práci je typický rozdiel micro – macro 0,2 až 0,4 (napr. 0,835 vs. 0,663 pri `hazard-category`), čo signalizuje slabšie výsledky na zriedkavých triedach. Práve preto je vhodné uvádzať obe metriky spolu.

2. Základný výpočet

```
from sklearn.metrics import f1_score

y_true = valid["hazard-category"]
y_pred = model.predict(valid["title"])

macro = f1_score(
    y_true, y_pred,
    average="macro", zero_division=0
)

micro = f1_score(
    y_true, y_pred,
    average="micro", zero_division=0
)

weighted = f1_score(
    y_true, y_pred,
    average="weighted", zero_division=0
)

print(f"Macro-F1:    {macro:.4f}")
print(f"Micro-F1:    {micro:.4f}")
print(f"Weighted-F1: {weighted:.4f}")
```

Dôležité:

- Poradie argumentov je `(y_true, y_pred)`. Pri F1 prehodenie mení precision a recall, a pri niektorých metrikách aj výsledok.
- `zero_division=0` explicitne určuje správanie pri nedefinovanom F1 a potláča varovania (vo vašej práci ho `compute_score` nemá nastavené).
- Scikit-learn priemeruje cez **zjednotenie tried** z `y_true` a `y_pred`. Ak model predikuje triedu, ktorá sa v referencii nevyskytuje, jej F1 je 0 a Macro-F1 klesá.

3. Hodnotenie po triedach

Samotné číslo Macro-F1 skrýva, ktoré triedy zlyhávajú. V práci sa importuje `classification_report`, ale nepoužíva sa.

```
from sklearn.metrics import classification_report

print(classification_report(
    y_true, y_pred,
    zero_division=0,
    digits=3
))
```

Výstup obsahuje precision, recall, F1 a support pre každú triedu, plus riadky `macro avg`, `weighted avg` (a `accuracy`, čo je pri single-label úlohe micro-F1).

Pre ďalšie spracovanie:

```
report = classification_report(
    y_true, y_pred,
    zero_division=0,
    output_dict=True
)
```

4. Matica zámen

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

ConfusionMatrixDisplay.from_predictions(
    y_true, y_pred,
    normalize="true",
    xticks_rotation=90,
    cmap="Blues",
    values_format=".2f",
)

plt.tight_layout()
plt.show()
```

Pri normalizácii po riadkoch (`normalize="true"`) treba pamätať, že riadky s malým počtom vzoriek (napr. 0,12; 0,25; 0,38) sú štatisticky nestabilné, ako uvádza aj vaša analýza chýb.

5. Kombinované skóre súťaže

Oficiálna metrika SemEval-2025 Task 9 je priemer macro-F1 pre `hazard` a pre `product`, pričom produkt sa hodnotí len na vzorkách so správne určeným nebezpečenstvom:

```
def compute_score(
    hazards_true,
    products_true,
    hazards_pred,
    products_pred
):
    # F1 pre hazard na vsetkych vzorkach
    f1_hazards = f1_score(
        hazards_true,
        hazards_pred,
        average="macro",
        zero_division=0
    )

    # F1 pre product len pri spravnom hazard
    mask = hazards_pred == hazards_true

    f1_products = f1_score(
        products_true[mask],
        products_pred[mask],
        average="macro",
        zero_division=0
    )

    return (f1_hazards + f1_products) / 2
```

Používajte pandas Series alebo NumPy polia (kvôli maskovaniu booleovským poľom).

Pre podúlohu 1 sa funkcia volá s `hazard-category` a `product-category`, pre podúlohu 2 s `hazard` a `product`.

6. Interval spoľahlivosti (bootstrap)

Pri malých množinách a veľa triedach je jedno číslo málo. Percentilový bootstrap:

```
import numpy as np

def bootstrap_f1(
    y_true, y_pred,
    n_boot=500,
    average="macro",
    seed=2024
):
    rng = np.random.default_rng(seed)

    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)

    n = len(y_true)
    scores = []

    for _ in range(n_boot):
        idx = rng.integers(0, n, n)

        scores.append(
            f1_score(
                y_true[idx],
                y_pred[idx],
                average=average,
                zero_division=0
            )
        )

    return np.percentile(scores, [2.5, 97.5])

lo, hi = bootstrap_f1(y_true, y_pred)

print(
    f"Macro-F1 = {macro:.3f} "
    f"95% CI [{lo:.3f}; {hi:.3f}]"
)
```

Pre porovnanie dvoch modelov použite **rovnaké bootstrap vzorky** (rovnaké `idx`) a počítajte interval pre **rozdiel** skóre, nie len dva samostatné intervaly.

Poznámka k anomálii pri LR

Pri logistickej regresii bol bodový odhad mimo intervalu spoľahlivosti.

Bootstrap macro-F1 má pri veľa vzácnych triedach systematické vychýlenie, pretože vzorka s opakovaním obsahuje v priemere len približne 63 % unikátnych záznamov.

Zriedkavé triedy preto v bootstrap vzorke miznú alebo majú menej príkladov. Percentilový interval potom môže ležať posunutý voči bodovému odhadu.

Je to realistické vysvetlenie, ktoré stojí za overenie, napríklad porovnaním s `BCa` intervalom alebo stratifikovaným bootstrapom.

7. Zhrnutie odporúčaní

1. Uvádzajte **Macro-F1 aj Micro-F1** spolu, ich rozdiel dáva informáciu o zriedkavých triedach.
2. Vždy nastavte `zero_division=0` a poradie `(y_true, y_pred)`.
3. Doplňte `classification_report` a maticu zámen, inak nevidno, ktoré triedy zlyhávajú.
4. Pri súťažnej úlohe použite oficiálne kombinované skóre S.
5. Vyhodnocujte na validačnej a testovacej množine, nie na dátach použitých pri tréningu.
6. Uvádzajte intervaly spoľahlivosti (bootstrap), najmä pri triedach s malým počtom vzoriek.

7. Pri nevyvážených triedach vyhodnoťte aj `class_weight='balanced'`, ale porovnávajte ho na viacerých množinách.

Prompt 3:

> Ako porovnať výsledky modelov LinearSVC a LogisticRegression pri použití rovnakej TF-IDF reprezentácie?

Relevantný výstup 3:

> **Porovnanie LinearSVC a LogisticRegression na rovnakej TF-IDF reprezentácii**
>
> **1. Zásady férového porovnania**
>
> - **Rovnaká reprezentácia:** identický `TfidfVectorizer` (rovnaké parametre) a rovnaký vstup (`title`).
> - **Rovnaké dáta:** trénovanie na tej istej trénovacej množine, vyhodnotenie na rovnakej validačnej a testovacej množine.
> - **Rovnaká metrika:** Macro-F1, Micro-F1 a skóre S.
> - **Rovnaké bootstrap vzorky** pre oba modely (inak sa rozdiel mieša so šumom).
>
> **2. Implementácia**

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score

def tfidf():
    return TfidfVectorizer(
        analyzer="char",
        ngram_range=(2, 5),
        strip_accents="unicode",
        max_df=0.5,
        min_df=5
    )

models = {
    "LinearSVC": LinearSVC(
        C=1.0,
        dual=True,
        max_iter=10000,
        random_state=2024
    ),
    "LogReg": LogisticRegression(
        max_iter=1000
    ),
}

results = {}

for name, clf in models.items():
    pipe = Pipeline([
        ("tfidf", tfidf()),
        ("clf", clf)
    ])

    pipe.fit(
        train["title"],
        train["hazard-category"]
    )

    pred = pipe.predict(valid["title"])
    results[name] = pred

    print(
        name,
        "macro:",
        round(
            f1_score(
                valid["hazard-category"],
                pred,
                average="macro",
                zero_division=0
            ), 3
        ),
        "micro:",
        round(
            f1_score(
                valid["hazard-category"],
                pred,
                average="micro",
                zero_division=0
            ), 3
        )
    )
```

3. Štatistické porovnanie (párový bootstrap)

```
import numpy as np

y = np.asarray(valid["hazard-category"])

pa = np.asarray(results["LinearSVC"])
pb = np.asarray(results["LogReg"])

rng = np.random.default_rng(2024)
diffs = []

for _ in range(500):
    # Rovnaka vzorka pre oba modely
    idx = rng.integers(0, len(y), len(y))

    diffs.append(
        f1_score(
            y[idx],
            pa[idx],
            average="macro",
            zero_division=0
        )
        -
        f1_score(
            y[idx],
            pb[idx],
            average="macro",
            zero_division=0
        )
    )

lo, hi = np.percentile(diffs, [2.5, 97.5])

print(
    f"Rozdiel Macro-F1: {np.mean(diffs):.3f} "
    f"[{lo:.3f}; {hi:.3f}]"
)
```

Ak interval rozdielu **neobsahuje 0**, rozdiel je štatisticky preukázaný.

4. Na čo si dať pozor

- **Hyperparametre:**

  LR s predvoleným `C=1` a SVC s `C=1` nie sú nutne rovnako vyladené.

  Férovejšie je ladiť `C` pre oba modely, napríklad pomocou `GridSearchCV`.

- **Nevyváženosť tried:**

  Vyskúšajte `class_weight="balanced"` pri oboch modeloch.

- **Po triedach:**

  Porovnajte `classification_report` oboch modelov, pretože celkové číslo môže skrývať rozdiely na zriedkavých triedach.

- **Viac množín:**

  Hodnoťte na validačnej aj testovacej množine (v práci sa víťaz zhoduje na oboch).

- **Pravdepodobnosti:**

  LR má `predict_proba`, LinearSVC nie.

  Použite `CalibratedClassifierCV`.

5. Zhrnutie

Rovnaký vektorizátor, rovnaké dáta, ideálne vyladené `C` pre oba modely, a rozdiel vyhodnotiť párovým bootstrapom s intervalom spoľahlivosti.

V tvojej práci LinearSVC výrazne prekonáva LR (S: 0,678 vs. 0,498), ale časť rozdielu môže pochádzať z nevyladeného `C` pri LR.

Predvolené `C=1` je pri riedkych znakových n-gramoch často príliš silná regularizácia.

2.3. Spôsob využitia výstupov

Relevantný výstup 1: Prvý výstup bol využitý pri implementácii modelu TF-IDF a LinearSVC, najmä pri nastavení znakových n-gramov, parametrov klasifikátora a spracovaní trénovacích dát. Získané odporúčania boli prispôsobené potrebám projektu. Správnosť implementácie bola overená spustením modelu a kontrolou získaných výsledkov.

Relevantný výstup 2: Druhý výstup bol využitý na pochopenie a správne vyhodnotenie klasifikačných modelov pomocou metrík Macro-F1 a Micro-F1. Získané informácie pomohli pri analýze výsledkov, interpretácii rozdielov medzi metrikami a hodnotení úspešnosti klasifikácie. Správnosť bola overená porovnaním vypočítaných metrík s výsledkami experimentov.

Relevantný výstup 3: Tretí výstup bol využitý pri porovnávaní modelov LinearSVC a LogisticRegression a pri analýze vplyvu váhovania tried na výsledky klasifikácie. Získané odporúčania pomohli pri interpretácii výsledkov a identifikácii silných a slabých stránok jednotlivých modelov. Závery boli overené porovnaním výsledkov na validačnej a testovacej množine.